

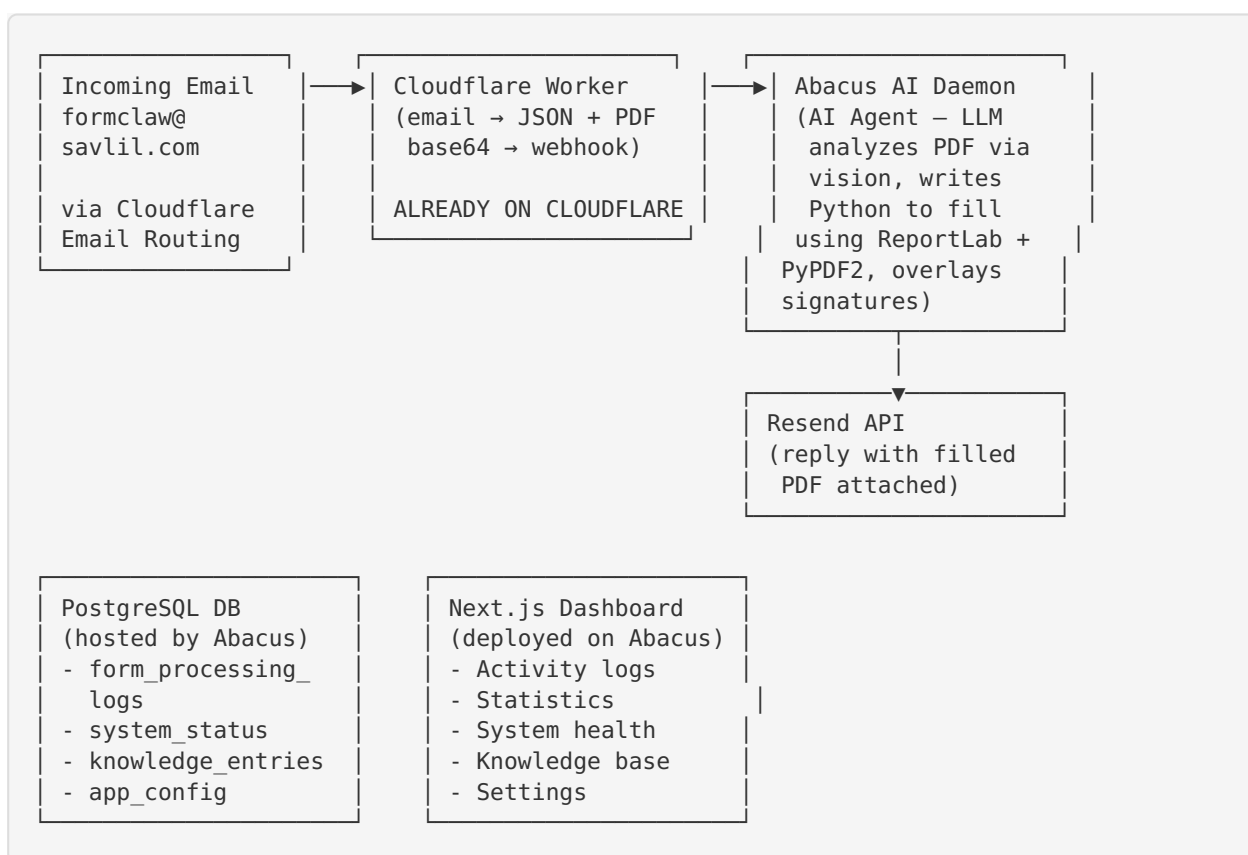
Form Claw — Cloudflare Workers Migration Plan

Date: 2026-07-09

Author: Form Claw Project

Status: DRAFT — awaiting review

1. Current Architecture (Abacus AI + Cloudflare hybrid)



Key insight: The “brain” is an LLM agent

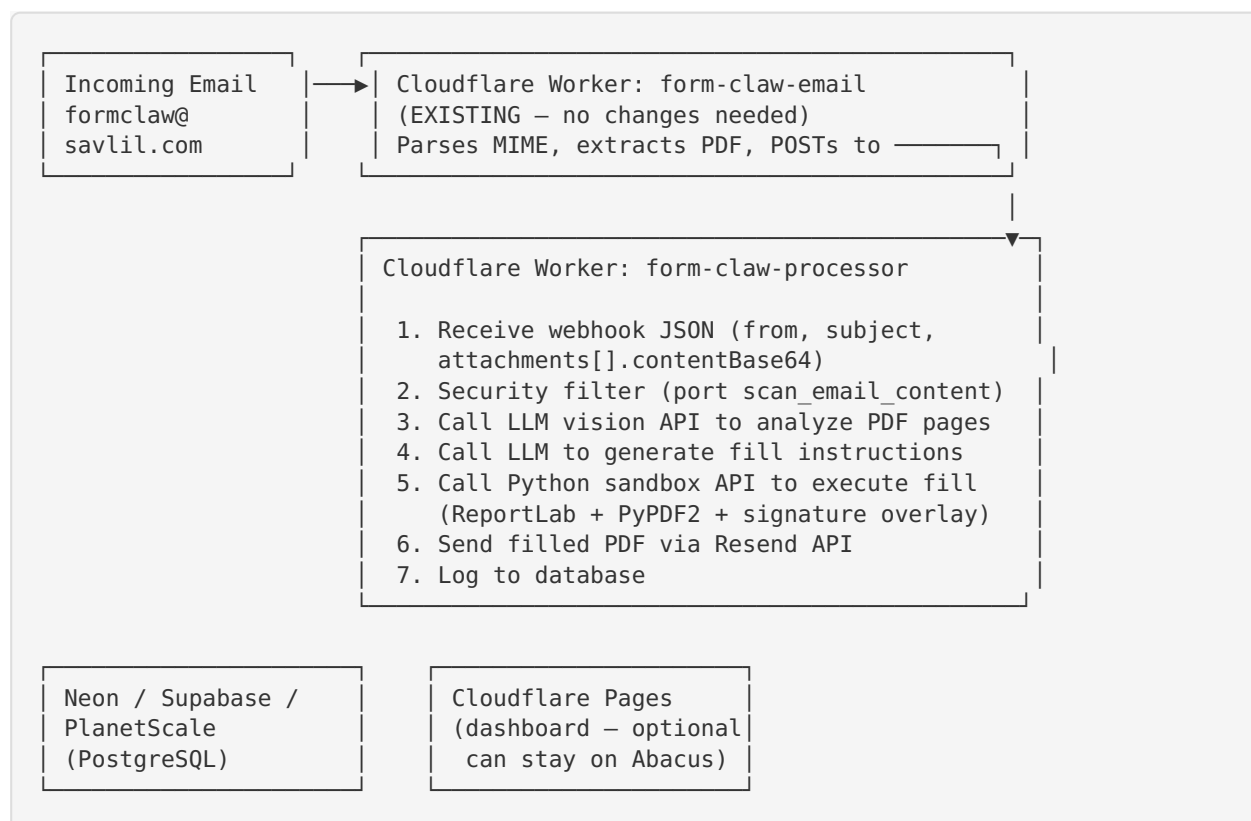
The form processor is NOT a deterministic script. It is an **AI agent** that:

1. Receives a PDF (base64) in the webhook payload
2. Uses **LLM vision** to analyze the PDF form (identify field labels, coordinates, checkboxes)
3. Maps fields to family data from the knowledge base
4. **Writes and executes Python code** at runtime using ReportLab + PyPDF2 to:
 - Place text at precise pixel coordinates
 - Handle Hebrew RTL text
 - Split ID numbers into individual digit boxes
 - Draw ellipses around selected options
 - Overlay transparent signature PNGs
5. Returns the filled PDF

This means the migration is NOT about porting a fixed codebase — it's about:

- Providing an LLM API that supports **vision** (PDF page → image analysis)
- Providing a **code execution sandbox** (Python with ReportLab, PyPDF2, Pillow)
- Providing **storage** for signatures, fonts, family data, and output PDFs

2. Target Architecture (fully on Cloudflare)



3. Component-by-Component Migration

3.1 Email Intake Worker (form-claw-email)

Status: ☒ Already on Cloudflare

Changes needed: Point `WEBHOOK_URL` to the new processor worker instead of Abacus daemon

Effort: Config change only (update Cloudflare secret)

3.2 Form Processor Worker (form-claw-processor) — THE HARD PART

This is the core challenge. The current processor is an AI agent with:

- Full Python runtime (ReportLab, PyPDF2, Pillow, tesseract)
- Multi-step LLM reasoning with vision
- File system access (read/write PDFs, PNGs)

Cloudflare Workers limitations:

- No Python runtime (Workers run V8/JavaScript/WASM)
- 30-second CPU time limit (free) / 15-minute wall-clock (paid, with Cron Triggers)

- No file system — must use R2 for storage
- Memory limit: 128MB

Proposed approach: The Worker becomes an **orchestrator** that calls external services:

```

form-claw-processor (JS/TS Worker)
├─ LLM API (vision + reasoning)
│   ├── Analyze PDF pages as images
│   ├── Determine field coordinates & values
│   └─ Generate fill instructions JSON
├─ PDF Processing API (external)
│   ├── Option A: Self-hosted microservice (e.g., on Fly.io/Railway)
│   │   └─ running Python with ReportLab/PyPDF2
│   ├── Option B: Use pdf-lib (JS) for simple fills + Cloudflare Workers
│   │   └─ ⚠ pdf-lib can't do: ReportLab overlays, Hebrew RTL shaping,
│   │       precise coordinate placement, transparent PNG overlay — this
│   │       would require significant rewriting
│   └─ Option C: Use a serverless function (AWS Lambda / GCP Cloud Run)
│       └─ with the Python fill logic
├─ R2 (Cloudflare object storage)
│   ├── Store signature PNGs, fonts, family_data.json
│   └─ Store filled PDFs temporarily
├─ Resend API (send reply — already working)
└─ Database (Neon Postgres — serverless, HTTP-based)
    ├── Log processing events
    └─ System status
  
```

3.3 Database

Current: Abacus-hosted PostgreSQL

Target: [Neon](https://neon.tech) (<https://neon.tech>) serverless PostgreSQL (free tier: 0.5GB storage, auto-suspend)

Why Neon: HTTP-based driver ([@neondatabase/serverless](https://github.com/neondatabase/serverless)) works inside Workers, auto-scales, free tier sufficient for this volume

Migration: Export schema + data from current DB, import into Neon

Alternative: Cloudflare D1 (SQLite) — simpler but requires schema rewrite from PostgreSQL syntax

3.4 Dashboard

Option A (recommended): Keep on Abacus — it works, it's free, minimal maintenance

Option B: Move to Cloudflare Pages — Next.js on Pages has some limitations (no `getServerSideProps` in edge runtime, must use Pages Functions). Would require:

- Converting auth to Cloudflare Access or custom JWT
- Replacing Prisma with Drizzle (works better with D1/Neon in edge)
- Significant effort for low value

Recommendation: Leave dashboard on Abacus, point it to the new Neon database.

3.5 Health Monitoring

- The `/api/health/check` route stays with the dashboard (wherever it lives)
- The Warning Investigator can run as a Cloudflare Cron Trigger
- Alerts continue via Resend API

4. LLM API Options

The form processor needs:

1. **Vision capability:** Send PDF pages as images, get field analysis
2. **Reasoning:** Multi-step thinking to map family data to detected fields
3. **Structured output:** Return JSON with field coordinates and values
4. **Hebrew language support:** Understand Hebrew form labels

Option A: Abacus AI ChatLLM API (current)

Endpoint: `https://apps.abacus.ai/api/v0/chat/completions`

Key: `ABACUSAI_API_KEY` (already configured)

Models: Routes to best available (GPT-4o, Claude, Gemini, etc.)

Vision: ☒ Supported via `modalities: ["image"]` or base64 image in messages

Hebrew: ☒ Excellent

Cost: Included in Abacus subscription (you're already paying)

Pros:

- Already configured and working
- No additional API key needed
- Model routing optimizes cost/quality

Cons:

- Dependency on Abacus AI platform
- Can't use if you leave Abacus entirely

Integration from Cloudflare Worker:

```
const response = await fetch('https://apps.abacus.ai/api/v0/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${env.ABACUSAI_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    messages: [
      {
        role: 'user',
        content: [
          { type: 'text', text: 'Analyze this PDF form...' },
          { type: 'image_url', image_url: { url: `data:image/png;base64,${pageImage-
Base64}` } }
        ]
      }
    ],
    model: 'gpt-4o' // or let RouteLLM choose
  })
});
```

Option B: xAI Grok API

Endpoint: `https://api.x.ai/v1/chat/completions`

Key: Get from console.x.ai (<https://console.x.ai>)

Models: `grok-2-vision-1212` (vision), `grok-2-1212` (text)

Vision:  Supported (OpenAI-compatible format)

Hebrew:  Good (not as extensively tested as GPT-4o for Hebrew)

Pricing (as of 2026):

Model	Input	Output
grok-2-vision	\$2.00 / 1M tokens	\$10.00 / 1M tokens
grok-2	\$2.00 / 1M tokens	\$10.00 / 1M tokens

Integration from Cloudflare Worker:

```
const response = await fetch('https://api.x.ai/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${env.XAI_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    model: 'grok-2-vision-1212',
    messages: [
      {
        role: 'user',
        content: [
          { type: 'text', text: 'Analyze this PDF form...' },
          { type: 'image_url', image_url: { url: `data:image/png;base64,${pageImage-Base64}` } }
        ]
      }
    ]
  })
});
```

Pros:

- OpenAI-compatible API (easy swap)
- Competitive pricing
- No platform lock-in

Cons:

- Requires separate API key & billing
- Hebrew OCR quality may vary vs GPT-4o
- Grok vision is newer, less battle-tested for precise coordinate extraction

5. The Python Sandbox Problem

This is the **biggest challenge**. The form filler needs Python with:

- `reportlab` — PDF overlay creation with precise coordinates
- `PyPDF2` — PDF merging
- `Pillow` — Signature image processing (RGBA transparency)
- Hebrew text shaping (RTL)

Cloudflare Workers cannot run Python natively.

Solutions:

5a. External Python Microservice (recommended)

Deploy a lightweight Python API on **Fly.io** (free tier: 3 shared VMs) or **Railway**:

```
# POST /fill-pdf
# Body: { original_pdf_b64, fill_instructions: [...], signature_b64, signer_id, date }
# Returns: { filled_pdf_b64 }
```

This service is stateless, receives the fill instructions (coordinates + values) from the LLM, executes the ReportLab/PyPDF2 code, and returns the filled PDF.

Cost: Free (Fly.io free tier) or ~\$3-5/month

Latency: ~2-5 seconds per fill

Code: Port the existing `fill_fnx_form.py` logic + the skill's ReportLab patterns

5b. pdf-lib (JavaScript, runs in Worker)

Use [pdf-lib](https://pdf-lib.js.org/) (https://pdf-lib.js.org/) for basic PDF manipulation inside the Worker itself.

Limitations:

- ❌ No Hebrew RTL text shaping (would need additional WASM shaping library)
- ❌ No transparent PNG overlay with the same quality as ReportLab
- ⚠️ Coordinate placement possible but less precise than ReportLab
- ❌ No ellipse drawing for OR/slash selection

Verdict: Not suitable for the current form complexity. Would require significant feature regression.

5c. Cloudflare Workers + Python (experimental)

Cloudflare now has **Python Workers** (beta). However:

- Limited package support (no ReportLab, no PyPDF2)
- Still in beta with restrictions
- Not production-ready for this use case

Recommendation: Use **5a** (external Python microservice on Fly.io).

6. Storage: Cloudflare R2

Replace local file system with R2:

Asset	Current Location	R2 Key
Ze'ev's signature	/home/ubuntu/shared/zr sig-nature nxp.png	assets/signatures/zeev.png
Keren's signature	/home/ubuntu/shared/keren sig.png	assets/signatures/keren.png
Hebrew font	/home/ubuntu/shared/fonts/FtPilKahol2.ttf	assets/fonts/Ft-PilKahol2.ttf
English font	/home/ubuntu/shared/fonts/Playzone.ttf	assets/fonts/Playzone.ttf
Family data	/home/ubuntu/shared/family_data.json	config/family_data.json
Filled PDFs	/home/ubuntu/shared/form-bot_work/	output/<timestamp>/

Cost: R2 free tier — 10GB storage, 10M reads/month, 1M writes/month. More than enough.

7. GitHub Integration & CI/CD

All worker code will be sourced from the **existing GitHub repo** (zeevrussak/form-claw).

Repository Structure (proposed additions)

```

form-claw/
├── workers/
│   ├── email-intake/
│   │   ├── src/index.ts
│   │   ├── wrangler.toml
│   │   └── package.json
│   └── form-processor/
│       ├── src/
│       │   ├── index.ts
│       │   ├── llm.ts
│       │   ├── security.ts
│       │   ├── pdf-service.ts
│       │   └── db.ts
│       ├── wrangler.toml
│       └── package.json
├── services/
│   └── pdf-filler/
│       ├── app.py
│       ├── requirements.txt
│       ├── Dockerfile
│       └── fly.toml
└── nextjs_space/

```

existing Cloudflare Email Worker

NEW: orchestrator worker

main handler

LLM API client (ChatLLM or Grok)

port of security_filter.py

calls external Python API

Neon database client

Python microservice (Fly.io)

existing dashboard (stays on Abacus)

Cloudflare Wrangler Configuration

```
# workers/form-processor/wrangler.toml
name = "form-claw-processor"
main = "src/index.ts"
compatibility_date = "2026-01-01"

[[r2_buckets]]
binding = "ASSETS"
bucket_name = "form-claw-assets"

[vars]
FAMILY_DATA_KEY = "config/family_data.json"
PDF_SERVICE_URL = "https://form-claw-pdf.fly.dev"
```

Cloudflare Secrets (via `wrangler secret put`)

Secret Name	Purpose	Source
RESEND_API_KEY	Send reply emails	Resend dashboard
DATABASE_URL	Neon PostgreSQL connection	Neon dashboard
LLM_API_KEY	LLM API authentication	See Option A or B below
PDF_SERVICE_TOKEN	Auth for Python microservice	Self-generated
HEARTBEAT_TOKEN	Dashboard health reporting	Existing value

Cloudflare Variables (non-secret, in `wrangler.toml`)

Variable	Value
WHITELISTED_SENDERS	k6622024@gmail.com,2396119@gmail.com,...
PDF_SERVICE_URL	https://form-claw-pdf.fly.dev
DASHBOARD_URL	https://form-claw.abacusai.app
LLM_PROVIDER	chatllm or grok

CI/CD via GitHub Actions

```
# .github/workflows/deploy-workers.yml
name: Deploy Cloudflare Workers
on:
  push:
    branches: [main]
    paths: ['workers/**']
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
          workingDirectory: workers/form-processor
```

8. Cost Estimate — 2 forms/day

Per-form processing breakdown

Each form fill involves approximately:

- **1 LLM vision call** (analyze 1-2 PDF pages as images): ~2,000 input tokens + ~1,500 output tokens per page
- **1 LLM text call** (generate fill instructions): ~3,000 input tokens + ~2,000 output tokens
- **1 Python API call** (fill PDF): CPU time ~3 seconds
- **1 Resend email** (reply with PDF)
- **1 DB write** (log entry)

Estimated tokens per form: ~5,000 input + ~3,500 output (single page form)

Monthly cost at 2 forms/day (60 forms/month)

Component	Option A: ChatLLM	Option B: Grok (xAI)
LLM API	Included in Abacus sub	~\$0.60 input + ~\$2.10 output = ~\$2.70/mo
Cloudflare Workers	Free tier (100K req/day)	Free tier
Cloudflare R2	Free tier (10GB)	Free tier
Cloudflare Email Routing	Free	Free
Neon PostgreSQL	Free tier (0.5GB)	Free tier
Resend API	Free tier (100 emails/day)	Free tier
Python microservice (Fly.io)	Free tier (3 VMs)	Free tier
GitHub	Free	Free
Total monthly	~\$0/mo (Abacus sub covers LLM)	~\$3/mo

Token math for Grok

Per form:

Vision: 2,000 input × \$2/1M = \$0.004 + 1,500 output × \$10/1M = \$0.015

Text: 3,000 input × \$2/1M = \$0.006 + 2,000 output × \$10/1M = \$0.020

Total per form: ~\$0.045

Monthly (60 forms): 60 × \$0.045 = \$2.70

Note: Multi-page forms or complex forms requiring multiple LLM rounds could double costs. Estimate assumes typical 1-page Israeli government/HMO forms.

9. Implementation Phases

Phase 1: Foundation (1-2 days)

- [] Create Neon PostgreSQL database, import schema
- [] Create R2 bucket, upload assets (signatures, fonts, family_data.json)
- [] Set up GitHub Actions for Cloudflare deployment

Phase 2: Python Microservice (1-2 days)

- [] Port `fill_fnx_form.py` and the skill's fill logic into a stateless API
- [] Deploy to Fly.io

- ☐ Test with known form + fill instructions

Phase 3: Processor Worker (2-3 days)

- ☐ Write `form-claw-processor` Worker:
- Webhook handler
- Security filter (port `security_filter.py` to TypeScript)
- LLM client (with provider switch: ChatLLM / Grok)
- PDF service client
- Resend client (port `send_resend_reply.py` logic)
- DB logging (Neon HTTP client)
- Heartbeat reporting
- ☐ Deploy and wire up email intake Worker

Phase 4: Dashboard Reconnection (1 day)

- ☐ Point dashboard to new Neon database
- ☐ Update health check endpoint for new architecture
- ☐ Test all dashboard pages with new data source

Phase 5: Cutover & Monitoring (1 day)

- ☐ Update email intake Worker's `WEBHOOK_URL` to processor Worker
- ☐ Run end-to-end test
- ☐ Monitor first 24h of production
- ☐ Disable old Abacus daemon

Total estimated effort: 6-9 days

10. Risks & Mitigations

Risk	Impact	Mitigation
Worker 30s CPU limit exceeded on complex forms	Processing fails	Use Cloudflare Workers Unbound (no CPU limit, \$0.02/M requests) or offload heavy work to Python service
Hebrew RTL quality degrades with Grok vs GPT-4o	Incorrect field fills	Test both providers with 5+ real forms before switching; keep ChatLLM as fallback
Fly.io free tier cold starts	Slow first fill after idle	Use Fly.io <code>min_machines_running = 1</code> (\$3/mo) or accept 2-3s delay
PDF page-to-image conversion needed for vision	Workers can't render PDFs	Do PDF→PNG conversion in the Python microservice; return images to Worker for LLM call
Neon free tier auto-suspends after 5min idle	First request slow	Accept ~1s cold start, or use Neon Pro (\$19/mo) for always-on

11. Decision Matrix

Question	Recommended Choice
LLM Provider	Start with ChatLLM (free, tested). Switch to Grok if leaving Abacus.
PDF Processing	External Python microservice on Fly.io
Database	Neon PostgreSQL (free tier, serverless, HTTP driver)
Storage	Cloudflare R2 (free tier, native to Workers)
Dashboard	Keep on Abacus (lowest effort, already working)
CI/CD	GitHub Actions → Wrangler

12. What Stays the Same

- ✓ Email intake Cloudflare Worker (already deployed)
- ✓ Resend for outbound email (already configured)
- ✓ GitHub repository as source of truth
- ✓ Sender whitelist logic
- ✓ Security filter logic (ported to TypeScript)
- ✓ Family data structure (family_data.json)
- ✓ Signature assets (PNGs with transparency)
- ✓ Form-filling skill logic (coordinate placement, ellipses, split-digit fields)

13. What Changes

- ↻ Form processor: Abacus AI daemon → Cloudflare Worker + Python microservice
- ↻ Database: Abacus-hosted PostgreSQL → Neon serverless PostgreSQL
- ↻ Asset storage: Local filesystem → Cloudflare R2
- ↻ LLM calls: Abacus built-in agent → Direct API calls (ChatLLM or Grok)
- ↻ Code execution: Abacus agent Python sandbox → Fly.io Python microservice